

Key Management and Distribution

MODULE V

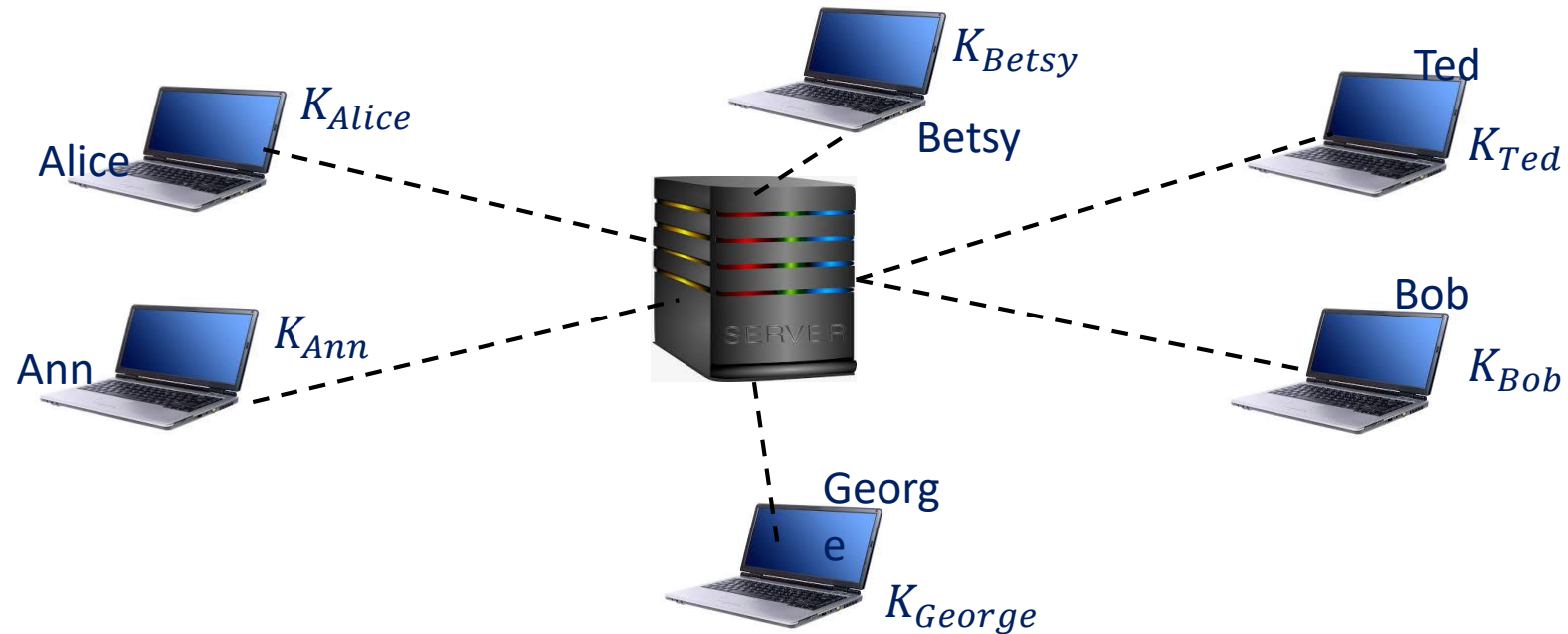
Introduction

- Symmetric key cryptography is more efficient than asymmetric key cryptography for enciphering large messages. However, it needs a shared secret key between two parties.
- If Alice needs to exchange confidential messages with N people, she needs N *different keys*.
- *If N people need to communicate with each other, a total of $N(N-1)$ keys is needed if we require that Alice and Bob use two keys for bi-directional communication.*
- Only $N(N-1)/2$ keys are needed if we allow a key to be used for both directions.
- This means that if one million people need to communicate with each other, each person has almost one million different keys, in total, almost one trillion keys are needed.
- This is normally referred to as the N^2 problem because the number of required keys for N entities is N^2 .
- The number of keys is not the only problem; the distribution of keys is another.

Key Distribution Center (KDC)

- A practical solution is the use of a trusted third party, referred to as a *key-distribution center (KDC)*. Each node establishes a shared secret key with the KDC.

How say Alice sends a confidential message to Bob?



A KDC is a trusted third-party entity responsible for securely distributing cryptographic keys between parties that wish to communicate securely. It acts as a central authority, managing the generation, distribution, and revocation of symmetric and asymmetric keys.

A Key Distribution Center (KDC) is a critical component in cryptography's Key Management concept, ensuring secure and efficient key exchange between parties.

KDC cntd..

- Alice sends a request to the KDC stating that she needs a session (temporary) secret key between Alice and Bob.
- The KDC informs Bob about the Alice's request.
- If Bob, agrees, a session key is created between the two.

The secret key between Alice and Bob that is established with the KDC is used to authenticate Alice and Bob to the KDC and to prevent Eve from Impersonating either of them.

Session Keys: A KDC creates a secret key for each member. This secret key can be used only between the member and the KDC, not between two members. If Alice needs to communicate secretly with Bob, she needs a secret key between herself and Bob. A KDC can create a session key between Alice and Bob, using their keys with the center. The keys of Alice and Bob are used to authenticate Alice and Bob to the center and to each other before the session key is established. After the communication, the session key no longer useful.

Key Functions of a KDC

- **Key Generation:** The KDC generates keys for secure communication between parties.
- **Key Distribution:** The KDC securely distributes keys to authorized parties.
- **Key Management:** The KDC manages key lifecycles, including revocation, renewal, and updating.
- **Authentication:** The KDC verifies the identity of parties requesting key exchange.
- **Secure Storage:** The KDC securely stores sensitive key information.

Advantages

- **Secure Key Exchange:** KDC ensures secure key exchange between parties.
- **Scalability:** Supports large-scale key distribution.
- **Efficient Key Management:** Centralized key management simplifies key lifecycle management.

Dis-advantages

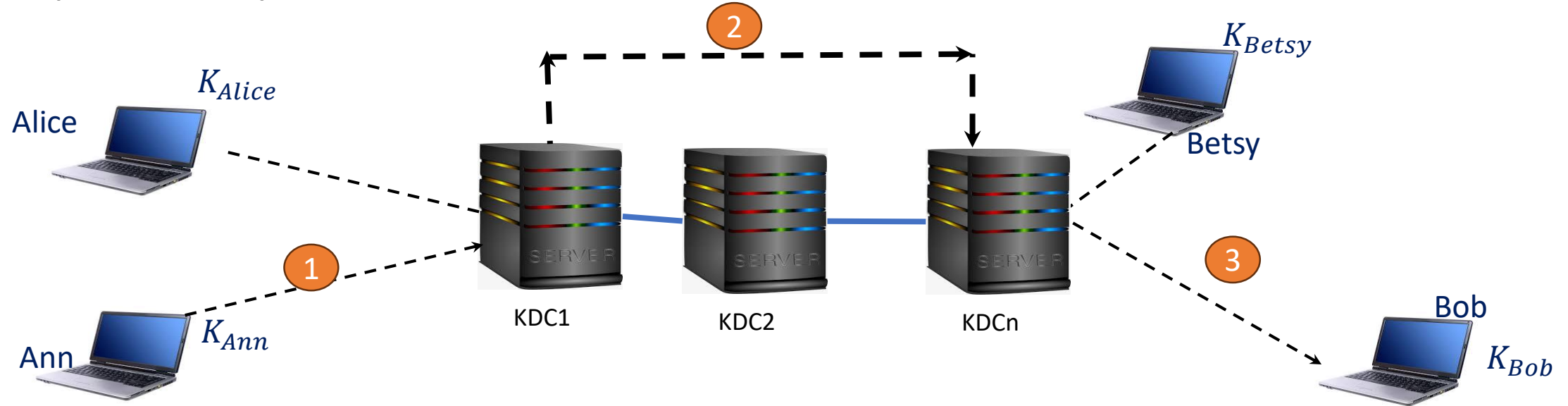
- **Single Point of Failure:** KDC becomes a single point of failure if compromised.
- **Key Escrow:** KDC has access to sensitive keys, potentially compromising security.
- **Performance Overhead:** KDC introduces additional latency and computational overhead

Real-World Implementations of KDC

- Kerberos
- Public Key Infrastructure (PKI)
- Secure Socket Layer / Transport Layer Security (SSL/TLS)

Flat Multiple KDCs

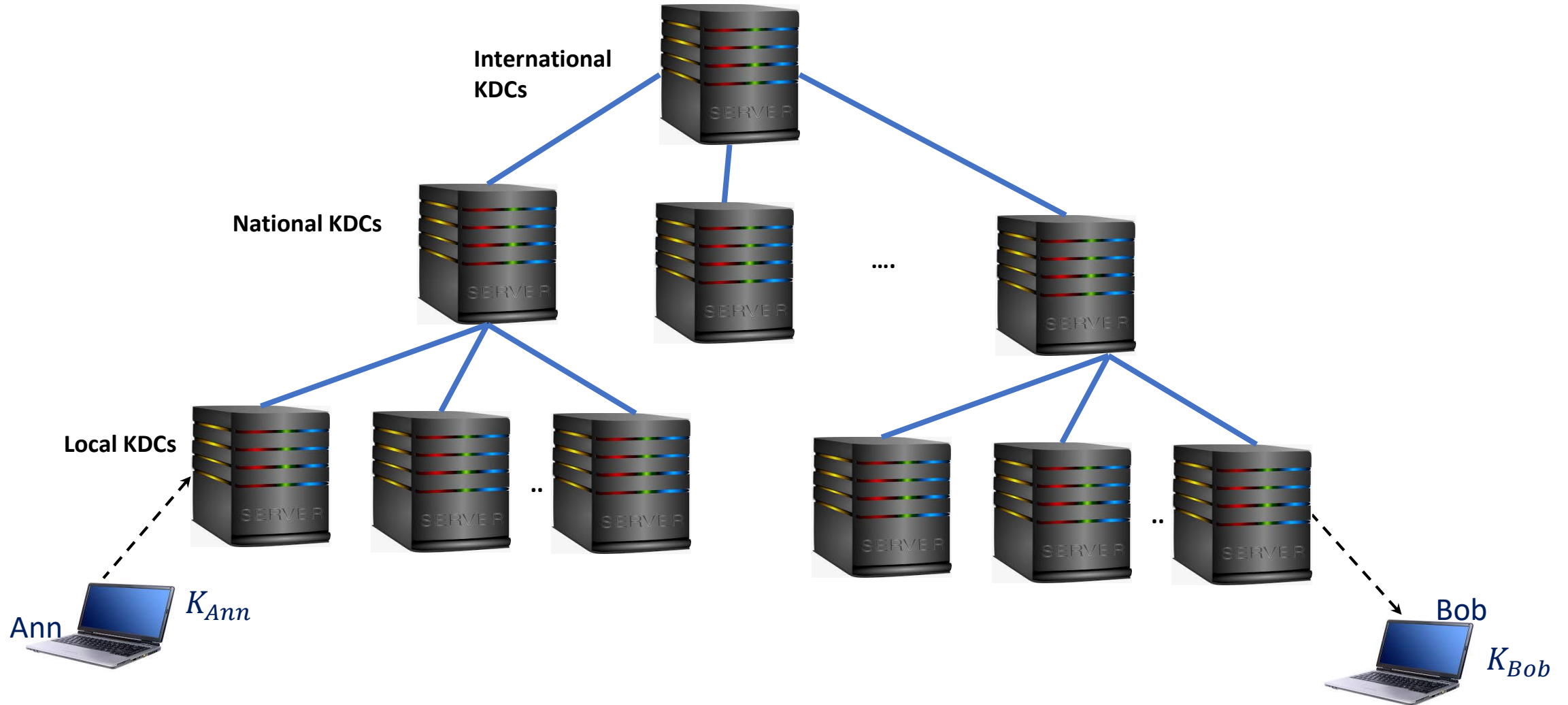
- When number of people using a KDC increases, the system becomes unmanageable, and a bottleneck can result. To solve the problem, we require multiple KDCs divided into different domains.



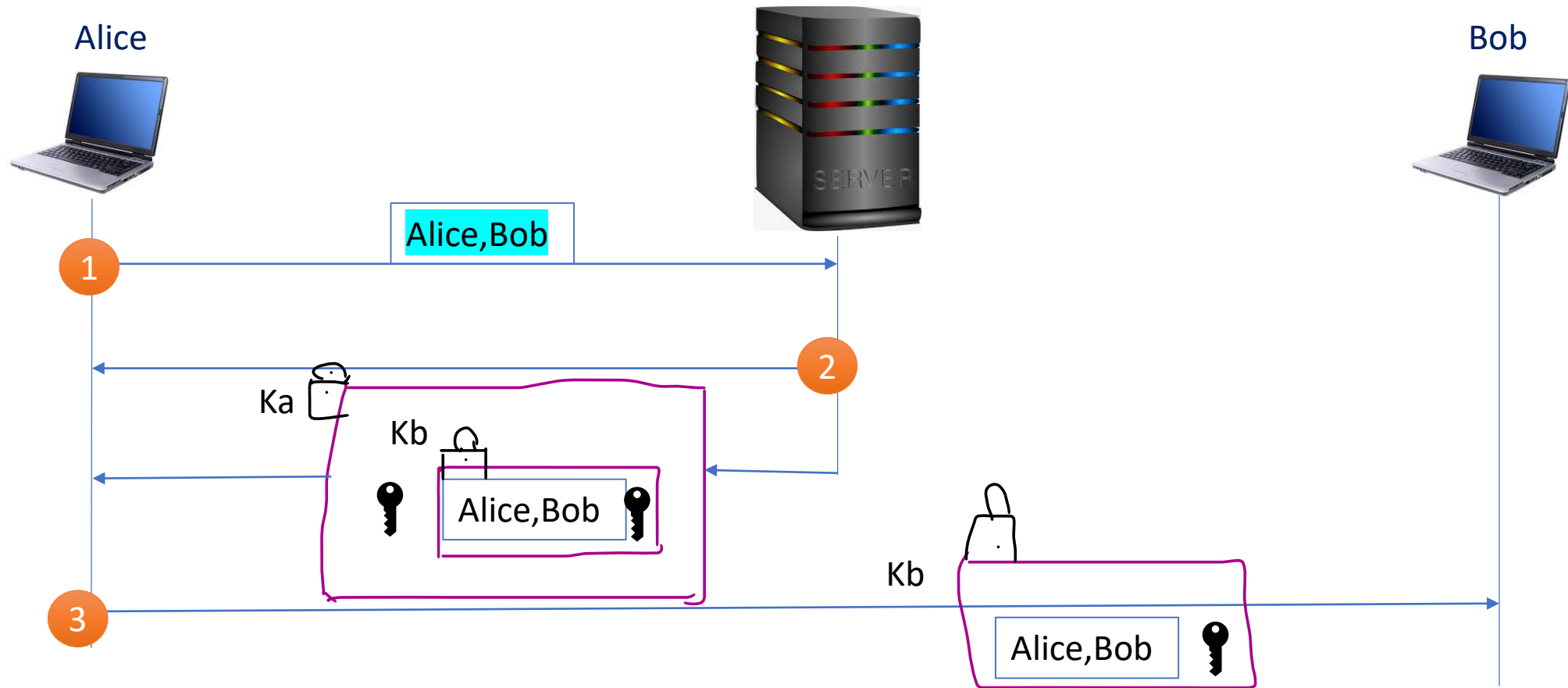
- If Ann wants to send a confidential message to Bob, who belongs to another KDC domain, Ann contacts her KDC, which in turn contacts the KDC in Bob's domain. The two KDCs can create a session key between Ann and Bob. Figure shows KDCs all at the same level. This is flat multiple KDCs.

Hierarchical Multiple KDCs

- The concept of flat multiple KDCs can be extended to a hierarchical system of KDCs, with one or more KDCs at the top of the hierarchy. For example, there can be local KDCs, national KDCs and International KDCs.



Simple protocol using a KDC



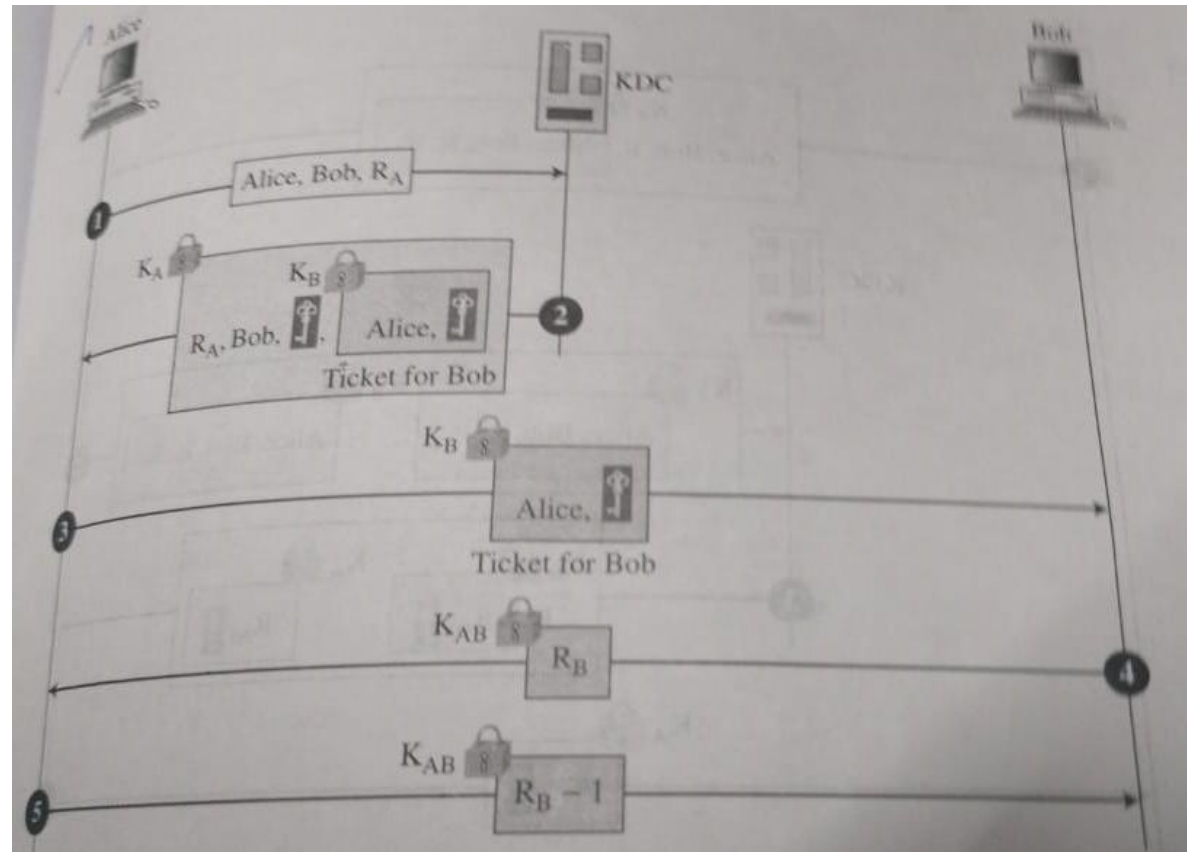
Simple protocol using a KDC cntd..

- 1. Alice sends a plaintext message to the KDC to obtain a symmetric session key between Bob and herself. The message contains her registered Identity and the Identity of Bob. This message is not encrypted, it is public. The KDC does not care.
- 2. The KDC receives the message and creates a ticket. The ticket is encrypted with Bob's key (K_b). The ticket contains the Identities of Alice and Bob and the session key (K_{ab}). The ticket with the copy of the session key is sent to Alice. Alice receives the message, decrypts it, and extracts the session key. She cannot decrypt Bob's ticket. The ticket is for Bob. This message contains double encryption. The ticket is encrypted and the entire message is also encrypted. In the second message, Alice is actually authenticated to the KDC, since Alice can only be able to decrypt the message using her secret key with KDC.
- 3. Alice sends the ticket to Bob. Bob opens the ticket and knows that Alice needs to send messages to him using K_{ab} as the session key. In this message, Bob is authenticated to KDC. Further, Bob is authenticated to Alice who trusts the KDC. In the same way, Alice is also authenticated to Bob, because Bob trusts the KDC has sent Bob the ticket that includes the Identity of Alice.

Drawback The Intruder can use the replay attack. An Intruder can save the message in step 3 and replay it later.

Needham-Schroeder Protocol

- The needham-Schroeder protocol is a foundation for many other protocols. It uses multiple challenge-response interactions between parties to achieve a flawless protocol. It uses two nonces: R_A and R_B .



Drawback

Needham-Schroeder Protocol cntd..

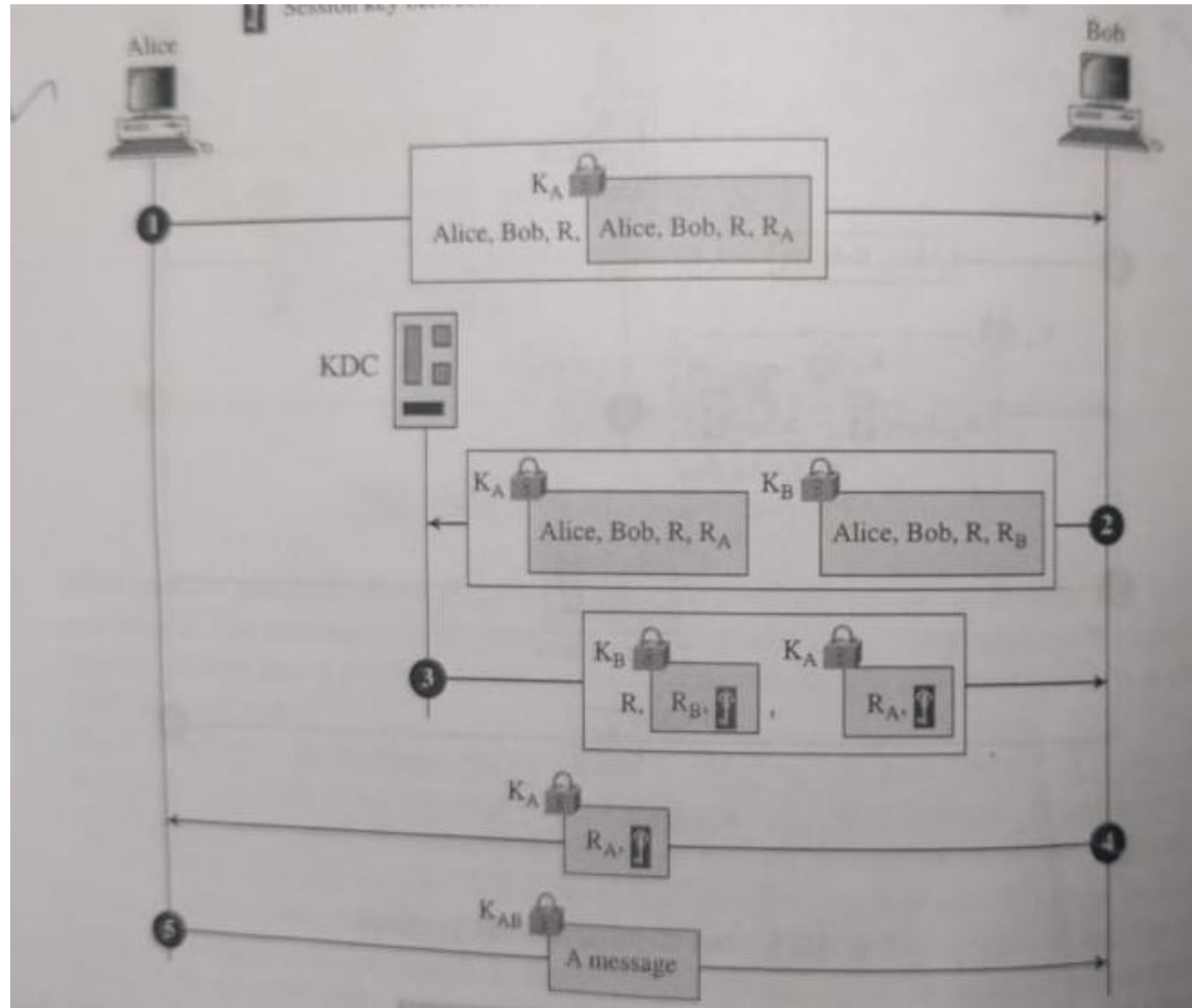
- Alice sends a message to the KDC that Includes her nonce, R_a , her Identity, and Bob's Identity.
- The KDC sends an encrypted message to Alice that Includes Alice's nonce, Bob's Identity, the session key, and an encrypted ticket for Bob. The whole message is encrypted with Alice's key.
- Alice sends Bob's ticket to him.
- Bob sends his challenge to Alice (R_b), encrypted with the session key.
- Alice responds to Bob's challenge. Note that the response carries R_{b-1} instead of R_b .

Otway-Rees protocol

This is another elegant protocol.

- Alice sends a message to Bob that includes a common nonce, R , the Identities of Alice and Bob, and a ticket for KDC that includes Alice's nonce R_a (a challenge for the KDC to use), a copy of the common nonce, R , and the Identities of Alice and Bob.
- Bob creates the same type of ticket, but with his own nonce R_b . Both tickets are sent to the KDC.
- The KDC creates a message that contains R , the common nonce, a ticket for Alice and a ticket for Bob, the message is sent to Bob. The tickets contain the corresponding nonce, R_a , or R_b , and the session key, K_{ab} .
- Bob sends Alice her ticket.
- Alice sends a short message encrypted with her session key K_{ab} to show that she has the session key.

Otway-Rees protocol

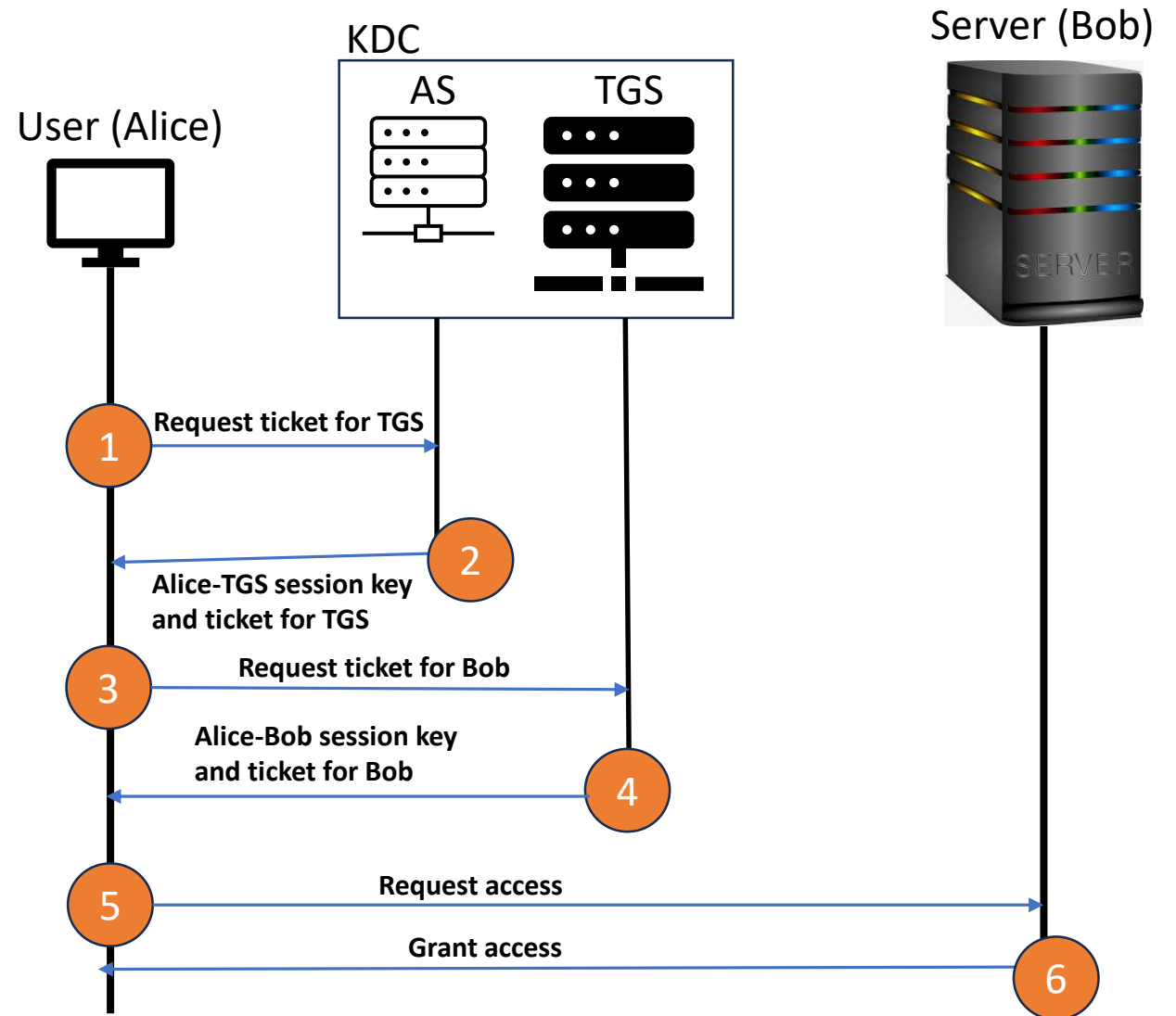


Kerberos

- Kerberos is an authentication protocol, and at the same time a KDC, that has become very popular. Several systems including Windows 2000, use Kerberos. It is named after the three headed dog in Greek mythology that guards the gates of Hades.
- Originally designed at MIT, it has gone through several versions. Here, we consider version 4, the most popular.

Servers

- The servers are involved in the Kerberos protocol: an authentication server (AS), a ticket-granting server (TGS), and a real (data) server that provides services to others. In the example, consider Bob is the real server and Alice is the user requesting service.



Servers cntd..

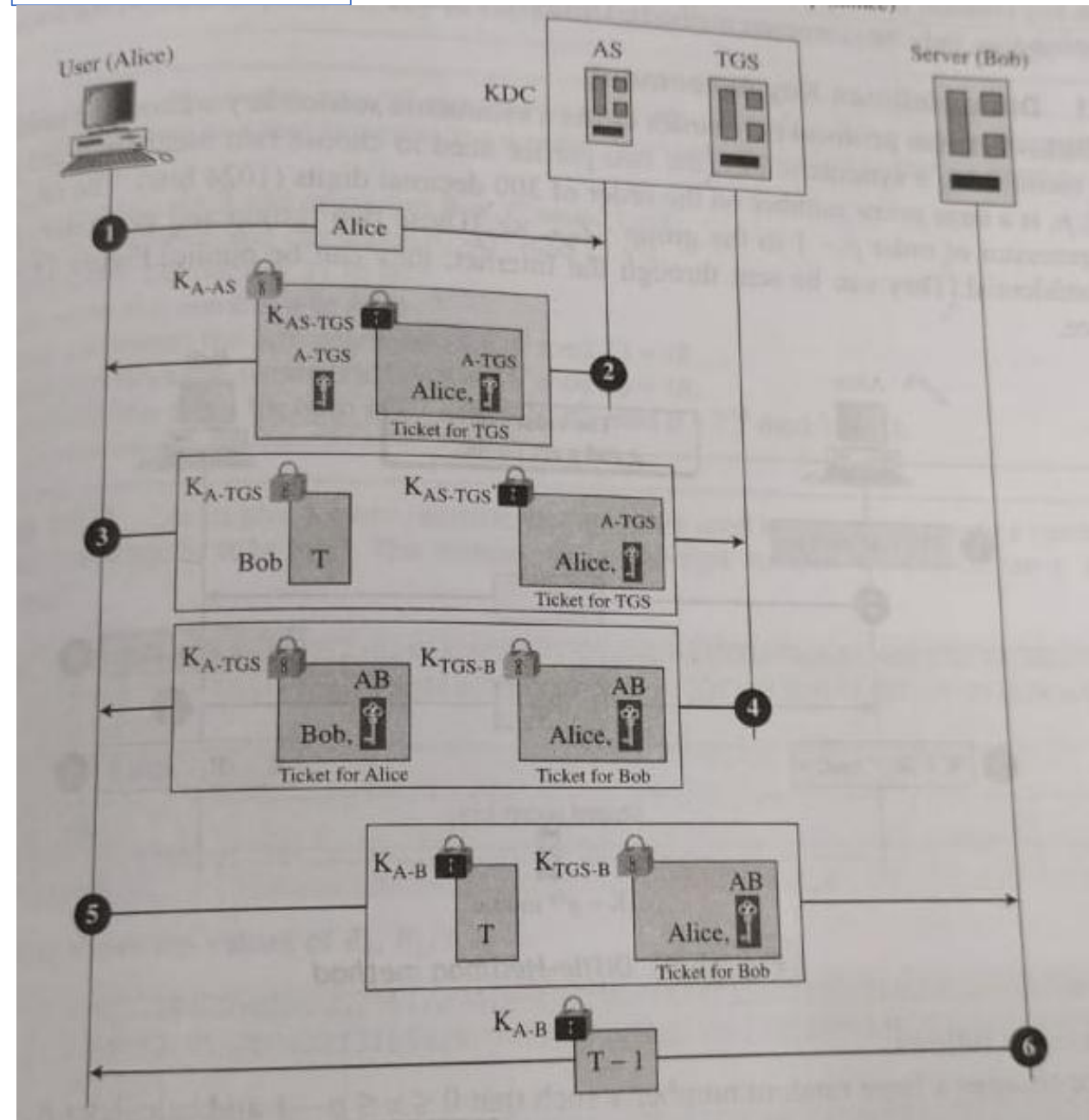
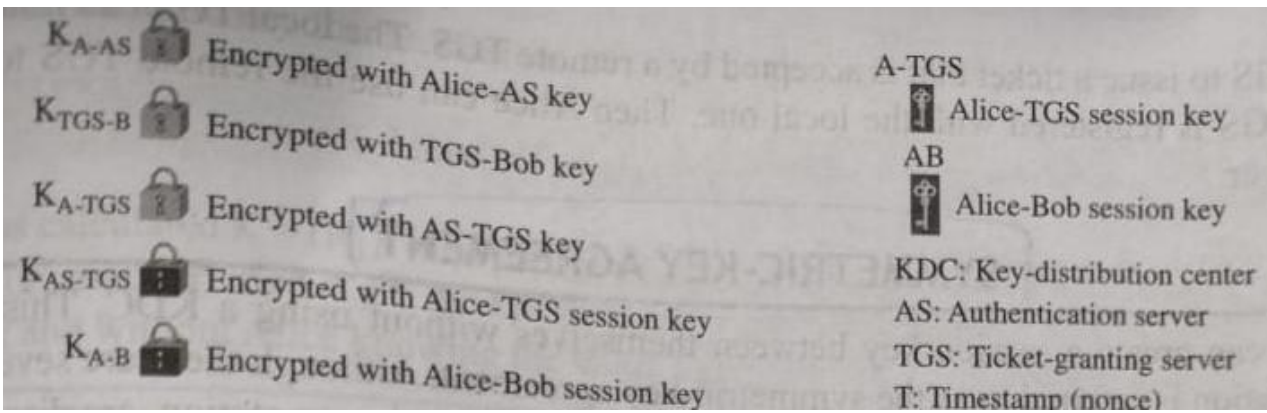
- **Authentication Server (AS):** The authentication server (AS) is the KDC in the Kerberos protocol. Each user registers with the AS and is granted a user identity and a password. The AS has a database with these identities and the corresponding passwords. The AS verifies the user, issues a session key to be used between Alice and the TGS, and sends a ticket for the TGS.
- **Ticket Granting Server (TGS):** The ticket granting server issues a ticket for the real server (Bob). It also provides the session key (K_{AB}) between Alice and Bob. Kerberos has separated user verification from the issuing of tickets. In this way, though Alice verifies her ID just once with the AS, she can contact the TGS multiple times to obtain tickets for different real servers.
- **Real Server:** The real server (Bob) provides services for the user (Alice). Kerberos is designed for a client-server program, such as FTP, in which a user uses the client process to access the server process. Kerberos is not used for person-to-person authentication.

Operation

- A client process (Alice) can access a process running on the real server (Bob) in six steps, as shown.
- 1. Alice sends her request to the AS in plain text using her registered identity.
- 2. The AS sends a message encrypted with Alice's permanent symmetric key (K_{A-AS}). The message contains two items: a session key, K_{A-TGS} , that is used by Alice to contact the TGS, and a ticket for the TGS that is encrypted with the TGS symmetric key, K_{AS-TGS} . Alice does not know K_{A-AS} , but when the message arrives, she types her symmetric password. The password and the appropriate algorithm together create K_{A-AS} if the password is correct. The password is then immediately destroyed; it is not sent to the network and it does not stay in the terminal. It is used only for a moment to create K_{A-AS} . The process now uses K_{A-AS} to decrypt the message sent. K_{A-TGS} and the ticket are extracted.
- 3. Alice now sends three items to the TGS. The first is the ticket received from the AS. The second is the name of the real server (Bob), the third is a timestamp that is encrypted by K_{A-TGS} . The time stamp prevents a replay by Eve.
- 4. Now, the TGS sends two tickets, each containing the session key between Alice and Bob, K_{A-B} . The ticket for Alice is encrypted with K_{A-TGS} ; the ticket for Bob is encrypted with Bob's key, K_{TGS-B} . Note that Eve cannot extract K_{AB} because Eve does not know K_{A-TGS} or K_{TGS-B} . She cannot replay step 3 because she cannot replace the timestamp with a new one (she does not know K_{A-TGS}). Even if she is very quick and sends the step3 message before the timestamp has expired, she still receives the same two tickets that she cannot decipher.
- 5. Alice sends Bob's ticket with the timestamp encrypted by K_{A-B} .
- 6. Bob confirms the receipt by adding 1 to the timestamp. The message is encrypted with K_{A-B} and sent to Alice.

Kerberos cntd..

Kerberos Example



Difference between version 4 and 5 of Kerberos

- Version 5 has a longer ticket lifetime.
- It allows tickets to be renewed.
- It can accept any symmetric-key algorithm
- It uses a different protocol for describing data types
- It has more overhead than version 4.

Symmetric Key Agreement

- Alice and Bob can create a session key between themselves without using a KDC.

Diffie-Hellman Key Exchange Algorithm



Select $X_a < p$
 $X_a = 3$.

$X_a = 3$.
 $A = g^{X_a} \bmod p$
 $A = 2^3 \bmod 13$
 $A = 8$

Agent X



$S = B^{X_a} \bmod p$
 $S = 11^3 \bmod 13$
 $S = 5$

secret
key

$[P = 13, g = 2]$

p is prime number, g is generator.
 g is the primitive root of p

$A = 8$

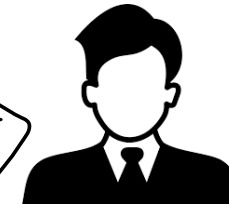
$B = 11$

public area

Select $Y_a < p$
 $Y_a = 7$.

$Y_a = 7$.
 $B = g^{Y_a} \bmod p$
 $B = 2^7 \bmod 13$
 $B = 11$

Agent Y



$S = A^{Y_a} \bmod p$
 $S = 8^7 \bmod 13$
 $S = 5$

secret
key

Key Exchange Algorithm



Spy

Can the hacker able to compute secret key by knowing p, g, A and B values. ?

Select $X_a < p$
 $X_a = 3$.

$X_a = 3$.
 $A = g^{X_a} \bmod p$
 $A = 2^3 \bmod 13$
 $A = 8$

Agent X



$S = B^{X_a} \bmod p$
 $S = 11^3 \bmod 13$
 $S = 5$

secret
key

$[p = 13, g = 2]$

p is prime number, g is generator.
g is the primitive root of p

$A = 8$

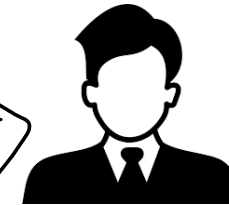
$B = 11$

public area

Select $Y_a < p$
 $Y_a = 7$.

$Y_a = 7$.
 $B = g^{Y_a} \bmod p$
 $B = 2^7 \bmod 13$
 $B = 11$

Agent Y



$S = A^{Y_a} \bmod p$
 $S = 8^7 \bmod 13$
 $S = 5$

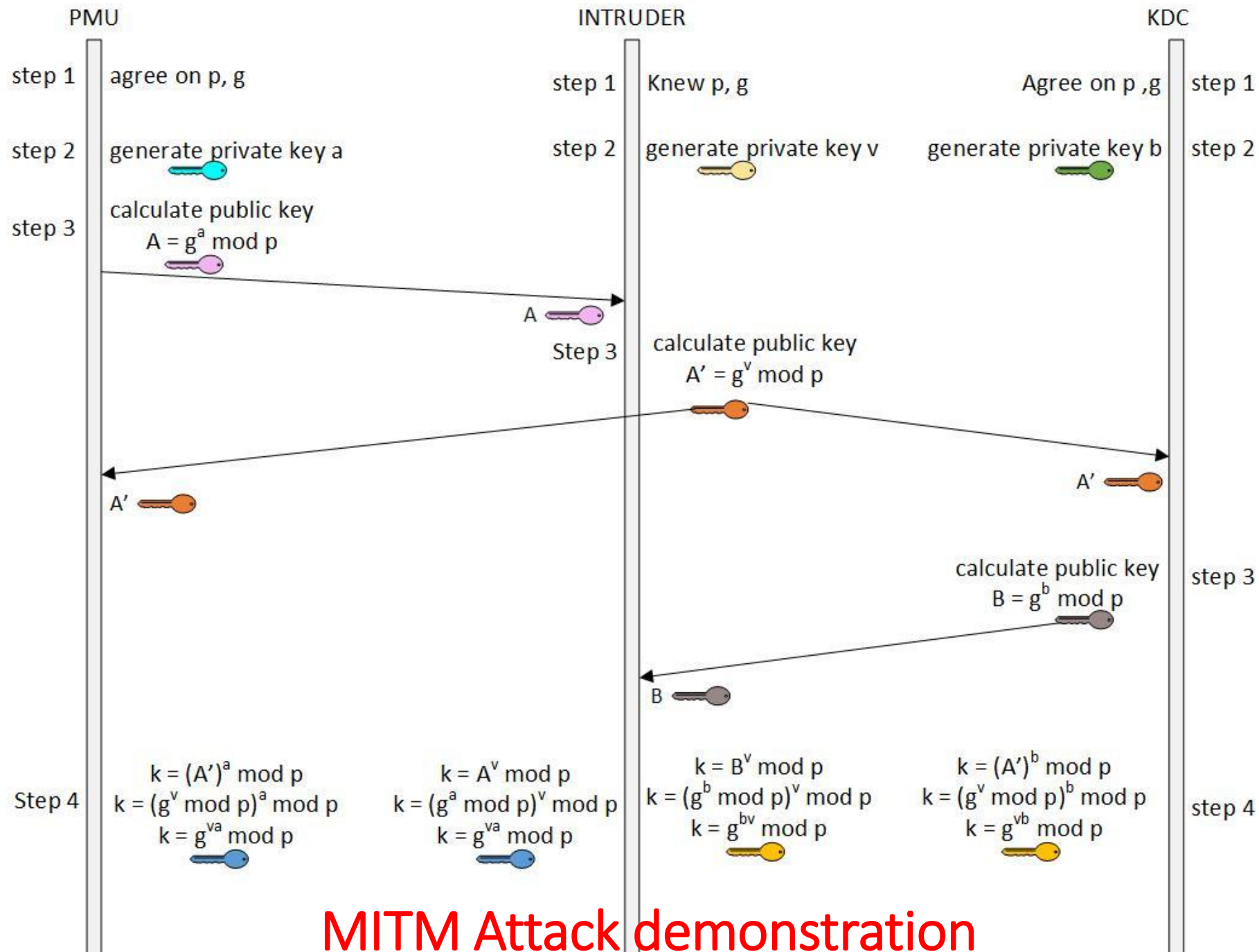
secret
key

Diffie-Hellman Key Exchange

- Published in 1976 by Diffie and Hellman.
- It allows two parties who have not previously met to securely establish a key which they can use to secure their communications.
- The Diffie-Hellman key exchange was the first widely used method of safely developing and exchanging over an insecure channel.
- These keys can then be used with symmetric key algorithms to transmit Information in a protected manner.

Steps in Diffie-Hellman Key Exchange

- Agent X and Agent Y, using insecure communication, agree on a huge prime p and a generator g .
- They don't care if someone listens in.
- Agent X chooses some large random integer $X_a < p$ and keeps it secret.
- Likewise, Agent Y chooses $Y_a < p$ and keeps it secret.
- These are their private keys.
- Agent X computes his "public key" $A = g^{X_a} \bmod p$ and sends it to Agent Y using insecure communication.
- Agent Y computes its "public key" $B = g^{Y_a} \bmod p$ and sends it to Agent X. Here, $0 < A < p$, $0 < B < p$.
- Agent X computes $S1 = B^{X_a} \bmod p$ and Agent Y computes $S2 = A^{Y_a} \bmod p$.

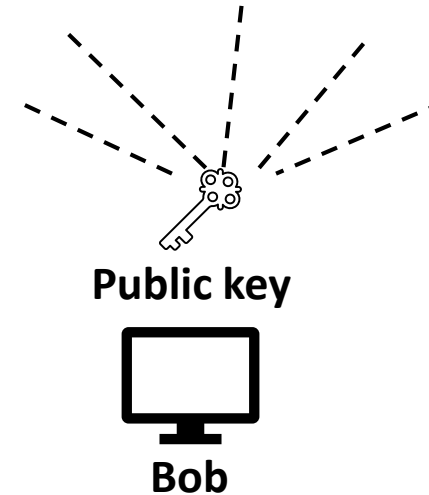


Public Key Distribution

- In asymmetric key cryptography, people do not need to know a symmetric shared key. If Alice wants to send a message to Bob, she only needs to know Bob's public key, which is open to the public and available to everyone. If Bob needs to send a message to Alice, he only needs to know Alice's public key, which is also known to everyone. In public key cryptography, everyone shields a private key and advertises a public key.
- Public keys like secret keys, need to be distributed to be useful. There are many ways to public keys can be distributed.

Public announcement

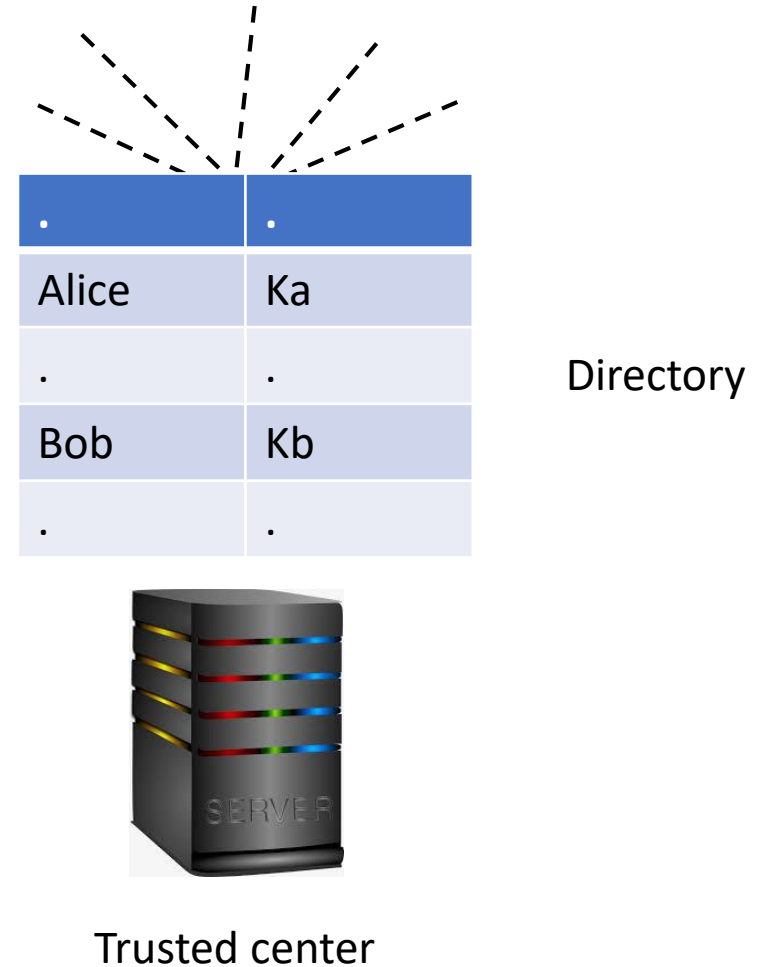
- The naïve approach is to announce public keys publicly. Bob can put his public key on his website or announce it in a local or national newspaper. When Alice needs to send a confidential message to Bob, she can obtain Bob's public key from his site or from the newspaper, or even send a message to ask for it.



- The approach is not secure. It is subject to forgery. For example, Eve could make such a public announcement. Before Bob can react, damage could be done. Eve can fool Alice into sending her a message that is intended to Bob. Eve could also sign a document with a corresponding forged private key and make everyone believe it was signed by Bob. The approach is also vulnerable if Alice directly requests Bob's public key. Eve can intercept Bob's response and substitute her own forged public key for Bob's public key. (MITM attack).

Trusted Center

- A more secure approach is to have a trusted center retain a directory of public keys. The directory, like the one used in a telephone system, is dynamically updated. Each user can select a private and public key, keep the private key, and deliver the public key for insertion into the directory. The center requires that each user register in the center and prove his or her Identity. The directory can be publicly advertised by the trusted center. The center can also respond to any inquiry about a public key.

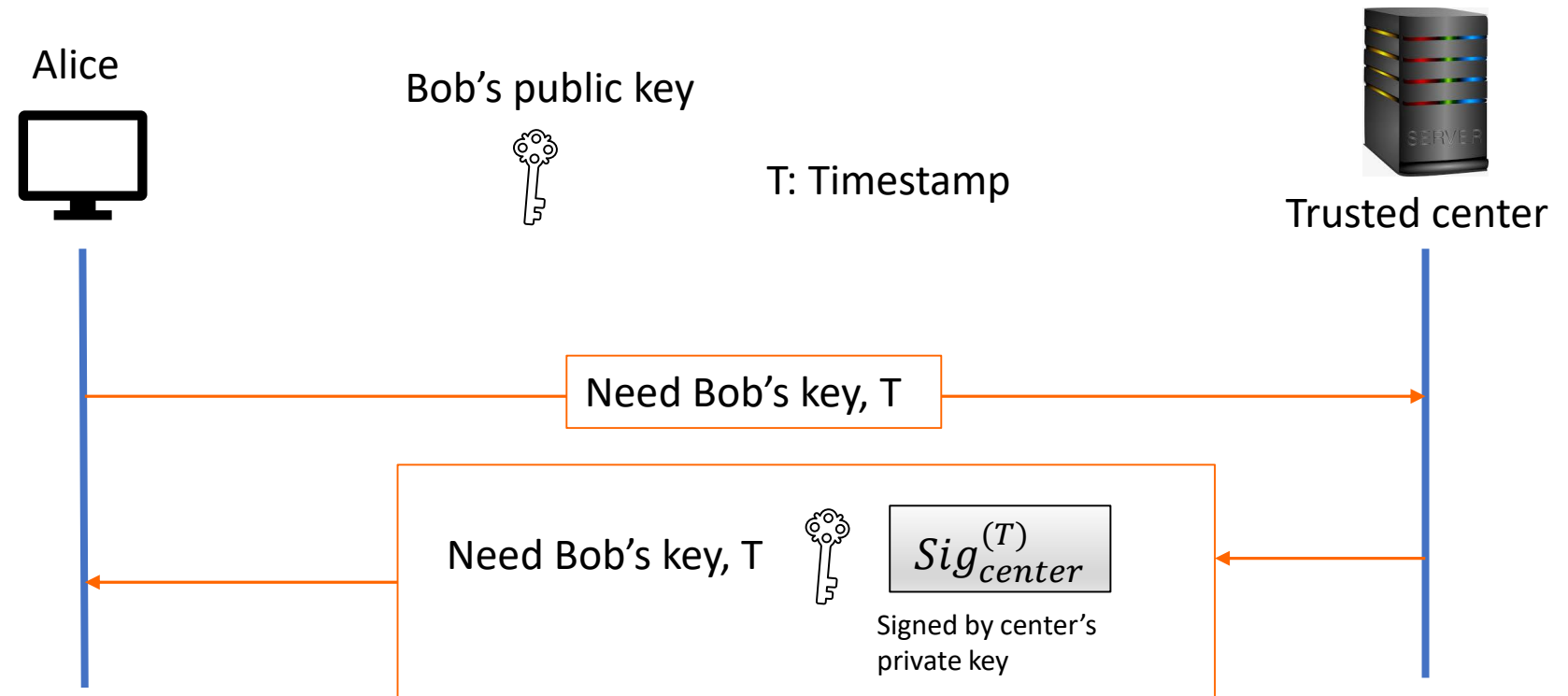


Controlled Trusted center

- A higher level of security can be achieved if there are added controls on the distribution of the public key. The public key announcements can include a timestamp and be signed by an authority to prevent interception and modification of the response. If Alice needs to know Bob's public key, she can send a request to the center including Bob's name and a timestamp. The center responds with Bob's public key, the original request, and the timestamp signed with the private key of the center. Alice uses the public key of the center, known by all, to verify the timestamp. If the timestamp is verified, she extracts Bob's public key.

.	.
Alice	Ka
.	.
Bob	Kb
.	.

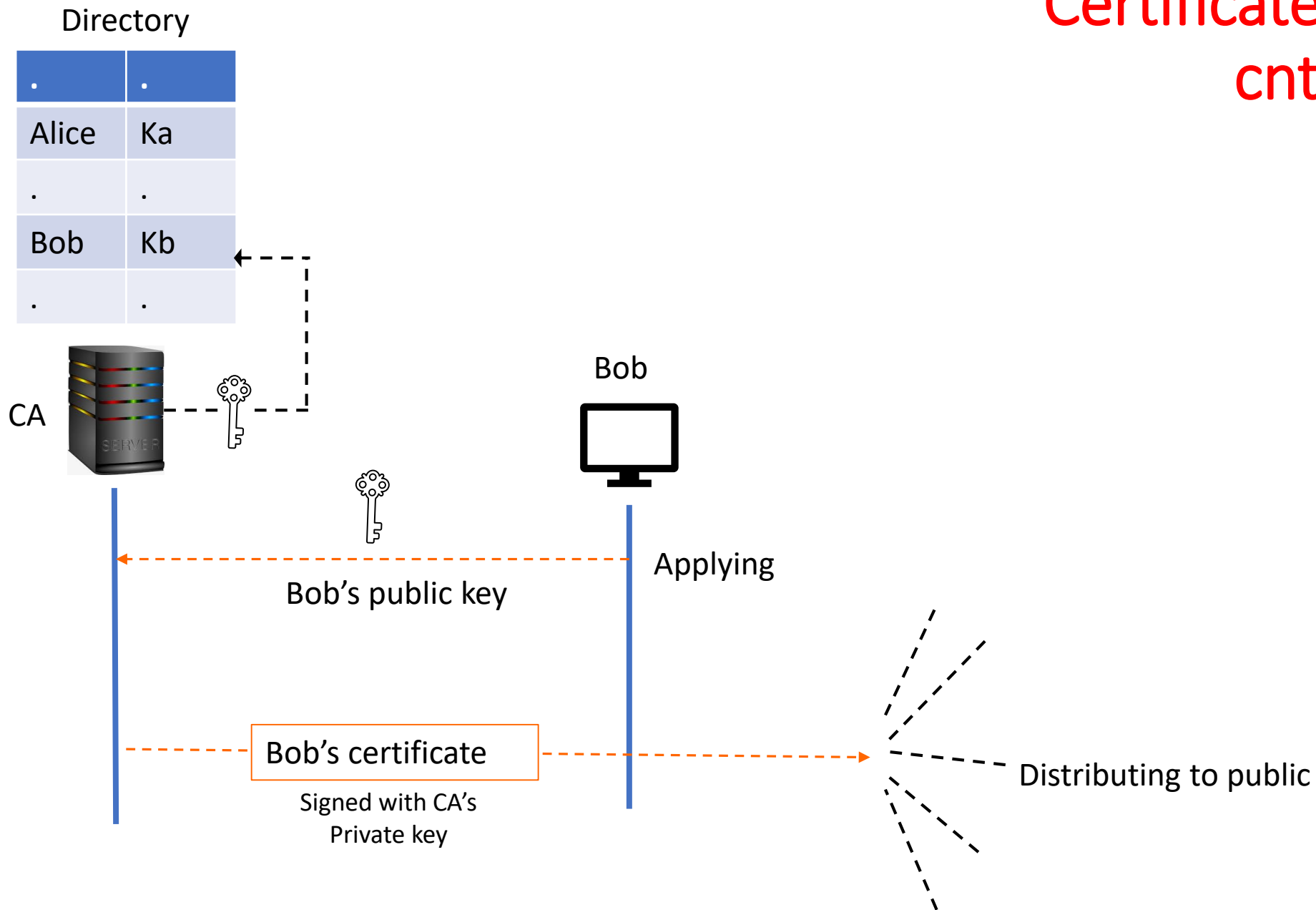
Directory



Certificate Authority

- The previous approach can create a heavy load on the center if the number of requests is large. The alternative is to create **public-key certificates**. Bob wants two things: he wants people to know his public key, and he wants no one to accept a forged public key as his. Bob can go to a **certification authority (CA)**, a federal or state organization that binds a public key to an entity and issues a certificate. The CA has a well-known public key itself that cannot be forged. The CA checks Bob's identification (using a picture ID along with other proof). It then asks for Bob's public key and writes it on the certificate. To prevent the certificate itself from being forged, the CA signs the certificate with its own private key. Now Bob can upload the signed certificate. Anyone who wants Bob's public key downloads the signed certificate and uses the center's public key to extract Bob's public key.

Certificate Authority cntd ..



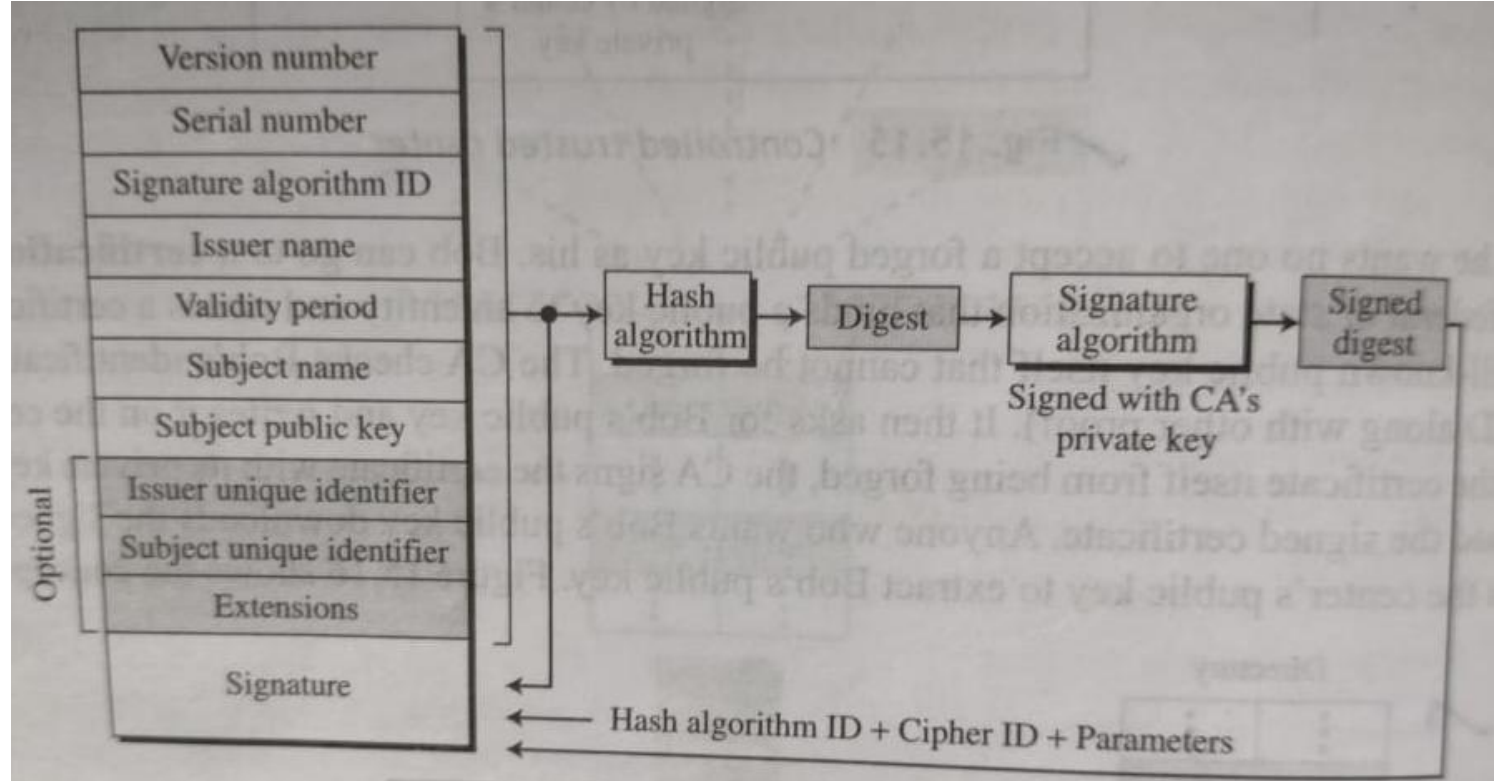
X.509

- Although the use of CA has solved the public key fraud, it has created a side effect. Each certificate may have a different format. If Alice wants to use a program to automatically download different certificates and digests belonging to different people, the program may not be able to do this. One certificate may have the public key in one format and another may have in different format. The public key may be on the first line in one certificate and on the third line in another. Anything that needs to be used universally must have a universal format.
- To remove this side effect, the ITU has designed **X.509**, a recommendation that has been accepted by the Internet with some changes. X.509 is a way to describe the certificate in a structured way. It uses a well-known protocol called ASN.1 (Abstract Syntax Notation.1) that defines fields familiar to C programmers.

X.509 Certificate fields

- A certificate has the following fields:

- Version number
- Serial number
- Signature algorithm ID
- Issuer Name
- Validity period
- Subject name
- Subject public key
- Issuer unique identifier
- Subject unique identifier
- Extensions
- Signature



X.509 Certificate fields cntd..

- A certificate has the following fields:
 - **Version number:** This field defines the version of X.509 of the certificate. The version number started at 0; the current version is 3.
 - **Serial number:** This field defines a number assigned to each certificate. The value is unique for each certificate issuer.
 - **Signature algorithm ID:** This field identifies the algorithm used to sign the certificate.
 - **Issuer Name:** This field identifies the certification authority that issued the certificate. The name is normally a hierarchy of strings that defines a country, a state, organization, department, and so on.
 - **Validity period:** This field defines the earliest time (not before) and the latest time (not after) the certificate is valid.
 - **Subject name:** This field defines the entity to which the public key belongs.
 - **Subject public key:** This field defines the owner's public key, the heart of the certificate. The field also defines the corresponding public-key algorithm (RSA, for example) and its parameters.
 - **Issuer unique identifier:** This optional field allows two issuers to have the same issuer field value, if the issuer unique identifiers are different
 - **Subject unique identifier:** This optional field allows two different subjects to have the same subject field value, if the subject unique identifiers are different.
 - **Extensions:** This optional field allows issuers to add more private information to the certificate.
 - **Signature:** This field is made of three sections. The first section contains all other fields in the certificate. The second section contains the digest of the first section encrypted with the CA's public key. The third section contains the algorithm identifier used to create the second section.

Certificate renewal

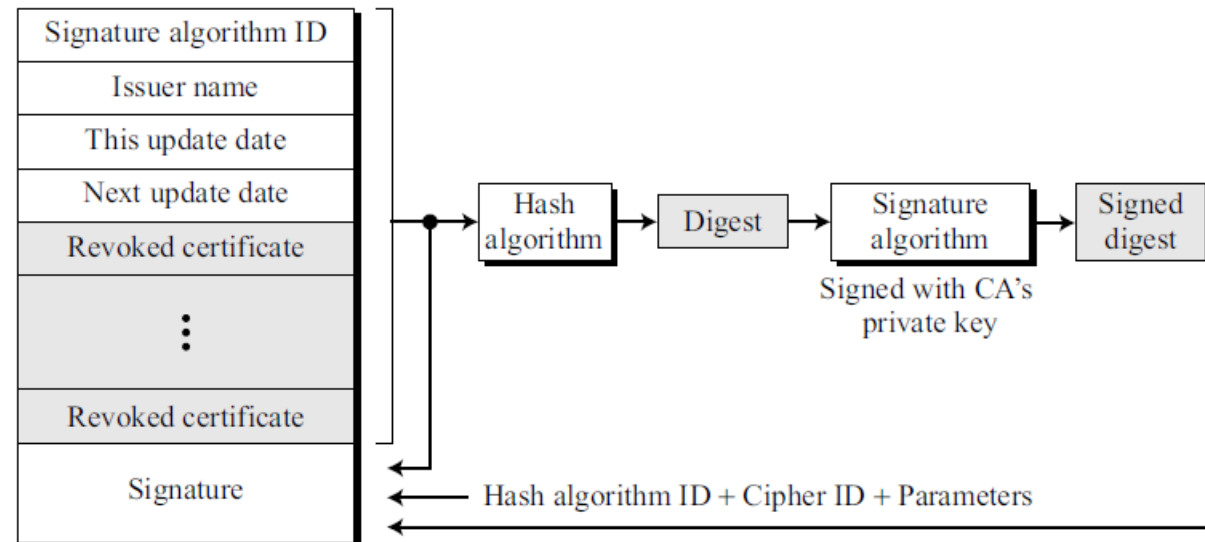
- Each certificate has a period of validity. If there is no problem with the certificate, the CA issues a new certificate before the old one expires.
- The process is like the renewal of credit cards by a credit card company; the credit card holder normally receives a renewed credit card before the one expires.

Certificate revocation

- In some cases a certificate must be revoked before its expiration.
 - The user's (subject's) private key (corresponding to the public key listed in the certificate) might have been comprised.
 - The CA is no longer willing to certify the user. For example, the user's certificate relates to an organization that she no longer works for.
 - The CA's private key, which can verify certificates, may have been compromised. In this case, the CA needs to revoke all unexpired certificates.
- The revocation is done by periodically issuing a certificate revocation list (CRL). The list contains all revoked certificates that are not expired on the date the CRL is issued. When a user wants to use a certificate, she first needs to check the directory of the corresponding CA for the last certificate revocation list.

Certificate Revocation List (CRL)

- A Certificate Revocation List has the following fields:
 - **Signature Algorithm ID**
 - **Issuer Name**
 - **This update date.** This field defines when the list is released.
 - **Next update date.** This field defines the next date when the new list will be released.
 - **Revoked Certificate.** This is a repeated list of all unexpired certificates that have been revoked. Each list contains two sections: user certificate serial number and revocation date.
 - **Signature.** This field is the same as the one in the certificate list



Delta Revocation

- To make revocation more efficient, the delta certificate revocation list (delta CRL) has been introduced. A delta CRL is created and posted on the directory if there are changes after this update date and next update date. For example, if CRLs are issued every month, but there are revocations in between, the CA can create a delta CRL when there is a change during the month. However, a delta CRL contains only the changes made after the last CRL.